



International School

Capstone Project 1

CMU-SE 450 – C1SE.30

Product Backlog

Version 1.1

Date: September 16th, 2026

TEST COVERAGE ANALYSIS AND AI-DRIVEN TEST GENERATION SYSTEM

Submitted by

Dinh, Huynh Tan

Phuc, Tran Huu

Quy, Nguyen Duy

Thuan, Ngo Huu

Approved by

Man, Nguyen Duc, Ph.D.

Proposal Review Panel Representative:

Name

Signature

Date

Capstone Project 1- Mentor:

Name

Signature

Date

PROJECT INFORMATION

Table 1 - Project Information

Project acronym	COVAI		
Project Title	Test Coverage Analysis and AI-Driven Test Generation System		
Start Date	10 Aug 2026	End Date	26 Dec 2026
Lead Institution	International School, Duy Tan University		
Project Mentor	Man, Nguyen Duc, Ph.D.		
Scrum master / Project Leader & contact details	Dinh, Huynh Tan Email: huynhtandinh.dev@gmail.com Tel: 0365 472 162 ID: 29219052500		
Partner Organization	Duy Tan University		
Project Web URL			
Team members	Name	Email	Tel
	Dinh, Huynh Tan	huynhtandinh.dev@gmail.com	0365472162
	Phuc, Tran Huu	phuc12435az@gmail.com	0842644169
	Quy, Nguyen Duy	quylavip333@gmail.com	0334814522
	Thuan, Ngo Huu	thuanhuugl@gmail.com	0385591447

DOCUMENT APPROVALS

The following signatures are required for approval of this document.

Table 2 - Document Approvals

Dinh, Huynh Tan Student ID: 29219052500 Scrum Master	Signature	Date
Phuc, Tran Huu Student ID: 29211458179 Team Member	Signature	Date
Quy, Nguyen Duy Student ID: 29211158823 Team Member	Signature	Date
Thuan, Ngo Huu Student ID: 29219065178 Team Member	Signature	Date

DOCUMENT NAME

Table 3 - Document Details

Document Title	Product Backlog Document		
Author(s)	Dinh, Huynh Tan; Ngo Huu Thuan		
Role	Dinh Huynh Tan, Ngo Huu Thuan		
Date	September 16th, 2026	File name:	C1SE.30_ProductBacklog_CovAI_ver1.1.docx

REVISION HISTORY

Table 4 - Revision History

Version	Person(s)	Date	Description
1.0	Thuan, Ngo Huu	22-Aug-2026	Create initial Product Backlog document based on Proposal
1.1	C1SE.30	16-Sep-2026	Final handoff revision aligned with Proposal v1.2, Project Plan v1.1, the two-module Admin/User model, High/Medium/Low priorities, canonical Sprint 1 task hours, and the current 33-story backlog.

TABLE OF CONTENTS

1.3. System Overview.....	7
2.1. Priority and Estimates.....	8
2.3. Product Backlog Specification (User)	11
5. References.....	17

LIST OF TABLES.....	
Table 1 - Project Information	2
Table 2 - Document Approvals	3
Table 3 - Document Details.....	4
Table 4 - Revision History	4
Table 5 - Priority Definition	8
Table 6 - Development Effort and Sprint Allocation	8
Table 7 - Sprint 1 Work Breakdown (Week 1 & Week 2)	9
Table 8 - Product Backlog Specification (Admin)	11
Table 9 - Product Backlog Specification (User).....	11
Table 10 - Constraints	15
Table 11 - Stakeholders and User Descriptions Summary	16
Table 12 - References	17

1. Introduction

The agile product backlog in Scrum is a prioritized features list, containing short descriptions of all functionality desired in the product. When applying Scrum, it's not necessary to start a project with a lengthy, upfront effort to document all requirements. Typically, a Scrum team and its product owner begin by writing down everything they can think of for agile backlog prioritization. This agile product backlog is almost always more than enough for a first sprint. The Scrum product backlog is then allowed to grow and change as more is learned about the product and its clients.

1.1 Purpose

- Provide a prioritized features list, containing short descriptions of all functionality desired in the CovAI product.
- Lists everything that the product owner and Scrum team feel should be included in the software they are developing for automated test coverage analysis, AI-driven test generation, and AI chatbot trustworthiness evaluation.

1.2 Scope

- The Product Backlog defines exactly two operational modules: Admin and User. Developers, QA engineers, testers, AI engineers, domain experts, and project owners use the same User module to test and evaluate their projects; no additional product module is created for these project personas.
- Specifies all functional requirements (FR-01 to FR-16) and non-functional quality attributes (NFR-01 to NFR-08) across the CovAI platform.
- Lists key capabilities across two core pillars: (1) Software Testing & Coverage Intelligence: ZIP/GitHub ingestion, isolated Docker sandbox execution, Statement/Branch/Function/Line coverage, AST & interactive CFG visualization, Cyclomatic Complexity analysis, Google Gemini AI test skeleton and full test generation, multi-framework support (Jest, Vitest, Supertest, Cypress, Playwright), GitHub webhooks, and real-time WebSocket

notifications; (2) AI Chatbot Trustworthiness Evaluation: Knowledge document ingestion, evaluation profile builder, scenario generation, multi-task evaluation runner (Knowledge Q&A, Usage Guidance, Action/Tool execution), tool/action safety verification, 3-layer trustworthiness scoring (Core Criteria, Task Module, Project Rules), diagnosis, and multi-version benchmark comparison.

- Provides concise descriptions of desired functionality and user value for each backlog item.
- Assigns three standard priority levels (High, Medium, Low) and development effort (hours) aligned with the 7 Sprints of the project schedule.

1.3 System Overview

CovAI is designed with a decoupled architecture comprising React 19 + Vite frontend, Express.js 5 REST API, PostgreSQL + Prisma ORM database, BullMQ + Redis job queues, Docker sandbox test execution, and Google Gemini API integration.

2. Product Backlog

2.1 Priority and Estimates

In accordance with Agile/Scrum empirical process control and CMU-SE 450 capstone guidelines, the Product Backlog features are evaluated and prioritized into three standard categories: High, Medium, and Low. Every feature is sized in committed man-hours and assigned to specific iterations across the 7 Sprints of the CovAI development lifecycle.

Table 5 - Priority Definition

Priority Level	Definition	Scope & Criteria	Target Sprints
High	Critical functionality required for the MVP and defense-ready core workflow.	Authentication, project management, source ingestion, Docker test execution, coverage, AI test generation, chatbot evaluation, dashboard, end-to-end testing, and deployment.	Sprint 1–7
Medium	Important analytical, automation, security, and operational capabilities that strengthen the MVP.	CFG/complexity analysis, multi-framework adapters, queue monitoring, GitHub webhook automation, ground-truth management, diagnosis/comparison, and security verification.	Sprint 3–7
Low	Useful non-blocking enhancements that can be deferred without preventing the core workflow.	Real-time WebSocket notifications and other supplementary presentation or convenience improvements.	Sprint 4

The CovAI engineering effort encompasses 7 Agile/Scrum Sprints comprising 1,920 development hours. The companion XLSX records the detailed acceptance criteria, remarks, status, type, project ID, sprint number, and numeric estimate for each of the 33 canonical user stories. Sprint 1 is reconciled directly to the Project Plan: 36 tasks, 336 hours total, with Week 1 = 168 hours and Week 2 = 168 hours.

Table 6 - Development Effort and Sprint Allocation

Sprint	Major Focus & Key Deliverables	Duration	Schedule Dates	Committed Hours
Sprint 1	Environment Setup, Authentication & Project Management	14 days	Sep 14, 2026 – Sep 27, 2026	336 hours

Sprint 2	Source Ingestion, Test Execution & Coverage Analysis	14 days	Sep 28, 2026 – Oct 11, 2026	336 hours
Sprint 3	Code Analysis, Multi-Test Framework Support & AI Test Generation	14 days	Oct 12, 2026 – Oct 25, 2026	336 hours
Sprint 4	GitHub Automation, Monitoring & System Integration	6 days	Oct 26, 2026 – Oct 31, 2026	144 hours
Sprint 5	AI Chatbot Trustworthiness Evaluation Module	14 days	Nov 08, 2026 – Nov 21, 2026	336 hours
Sprint 6	End-to-End System Testing & Quality Assurance	7 days	Nov 22, 2026 – Nov 28, 2026	168 hours
Sprint 7	System Deployment, Final Fixes & Project Delivery	11 days	Nov 29, 2026 – Dec 09, 2026	264 hours
Total Development	Total Development Effort across 7 Sprints (4 members @ 6h/day)	80 days	Sep 14, 2026 – Dec 09, 2026	1,920 hours
Total Project	Total Committed Academic Man-Hours (including Initial Phase & Defense)	107 days	Aug 10, 2026 – Dec 26, 2026	2,568 hours

Sprint 1 is structured into two 1-week cycles totaling 336 man-hours (24 man-hours/day across 4 team members). Week 1 (14/09/2026 - 20/09/2026) delivers 168 man-hours focused on UI prototyping, database schema engineering, front-end views, and foundational back-end controllers. Week 2 (21/09/2026 - 27/09/2026) delivers 168 man-hours completing back-end API services, authentication middleware, Docker host setup, and comprehensive unit test suites. Table 7 outlines the complete task breakdown matching the verified Project Plan.

Table 7 - Sprint 1 Work Breakdown (Week 1 & Week 2)

Week	Component	No	Task Name	Responsible Member	Estimate
Week 1	Design UI	1	UI Home Page	Thuan, Ngo Huu	8 hours
Week 1	Design UI	2	UI Registration	Phuc, Tran Huu	8 hours
Week 1	Design UI	3	UI Login	Dinh, Huynh Tan	8 hours
Week 1	Design UI	4	UI Landscape	Dinh, Huynh Tan	8 hours
Week 1	Design UI	5	UI Forgot Password	Quy, Nguyen Duy	8 hours
Week 1	Design UI	6	UI Profile User	Phuc, Tran Huu	8 hours
Week 1	Design UI	7	UI GitHub Repository	Thuan, Ngo Huu	8 hours
Week 1	Design UI	8	UI Project Management	Thuan, Ngo Huu	8 hours
Week 1	Design UI	9	UI Snapshot Management	Phuc, Tran Huu	8 hours
Week 1	Database	10	Design database (PostgreSQL & Prisma ORM)	Quy, Nguyen Duy / Thuan, Ngo Huu	24 hours

Week 1	Front end	11	(Front-end) Homepage	Quy, Nguyen Duy	8 hours
Week 1	Front end	12	(Front-end) Landscape	Quy, Nguyen Duy	8 hours
Week 1	Front end	13	(Front-end) Registration	Thuan, Ngo Huu	8 hours
Week 1	Front end	14	(Front-end) Login	Phuc, Tran Huu	8 hours
Week 1	Front end	15	(Front-end) Forgot Password	Dinh, Huynh Tan	8 hours
Week 1	Front end	16	(Front-end) GitHub Repository	Phuc, Tran Huu	8 hours
Week 1	Front end	17	(Front-end) Project Management	Quy, Nguyen Duy	8 hours
Week 1	Back end	18	(Back-end) Forgot Password	Dinh, Huynh Tan	16 hours
Week 1 Subtotal	Week 1 Total Effort (14/09/2026 - 20/09/2026: 7 days @ 24h/day)	-	-	All 4 Members	168 hours
Week 2	Back end	19	(Back-end) Reset Password	Dinh, Huynh Tan	16 hours
Week 2	Back end	20	(Back-end) Registration	Phuc, Tran Huu	16 hours
Week 2	Back end	21	(Back-end) Login & Session	Thuan, Ngo Huu	16 hours
Week 2	Back end	22	(Back-end) User Profile & PAT	Quy, Nguyen Duy	16 hours
Week 2	Back end	23	(Back-end) GitHub Repository	Dinh, Huynh Tan	16 hours
Week 2	Back end	24	(Back-end) Environment Setup & Docker Host	Phuc, Tran Huu	16 hours
Week 2	Back end	25	(Back-end) Authentication Middleware	Thuan, Ngo Huu	16 hours
Week 2	Back end	26	(Back-end) Snapshot Management	Quy, Nguyen Duy	16 hours
Week 2	Testing	27	Design Test Case for Registration	Dinh, Huynh Tan	4 hours
Week 2	Testing	28	Design Test Case for Login	Phuc, Tran Huu	4 hours
Week 2	Testing	29	Design Test Case for Forgot Password	Thuan, Ngo Huu	4 hours
Week 2	Testing	30	Design Test Case for Reset Password	Phuc, Tran Huu	4 hours
Week 2	Testing	31	Design Test Case for Profile	Dinh, Huynh Tan	4 hours
Week 2	Testing	32	Design Test Case for Homepage	Thuan, Ngo Huu	4 hours
Week 2	Testing	33	Design Test Case for Landscape	Quy, Nguyen Duy	4 hours

Week 2	Testing	34	Design Test Case for GitHub Repository	Dinh, Huynh Tan	4 hours
Week 2	Testing	35	Design Test Case for Project Management	Quy, Nguyen Duy	4 hours
Week 2	Testing	36	Design Test Case for Snapshot Management	Phuc, Tran Huu	4 hours
Week 2 Subtotal	Week 2 Total Effort (21/09/2026 - 27/09/2026: 7 days @ 24h/day)	-	-	All 4 Members	168 hours
Sprint 1 Total	Complete Sprint 1 Committed Work (36 tasks across 14 calendar days)	-	-	All 4 Members	336 hours

2.2 Product Backlog Specification (Admin)

Table 8 - Product Backlog Specification (Admin)

ID	Theme	As a/an	I want to	So that	Priority
PB-20	Asynchronous BullMQ Queue & Worker Monitoring [Sprint 4 40h]	Admin	Manage and monitor BullMQ background job queues and worker execution states in real-time	I can ensure smooth asynchronous analysis, detect queue bottlenecks, and retry failed tasks	Medium
PB-31	Security Hardening, Quotas & Container Sandboxing Verification [Sprint 6 72h]	Admin	Conduct security penetration checks, rate-limit verifications, and Docker sandbox escape audits	The platform is robustly defended against malicious code execution, token depletion, and unauthorized data access	Medium
PB-32	Production Deployment & Docker Compose Multi-Container Orchestration [Sprint 7 144h]	Admin	Deploy the complete CovAI platform using Docker Compose multi-container orchestration	The client, server, Redis, and PostgreSQL services run reliably in production for defense demo and user trials	High
PB-33	Admin Platform Oversight, Audit Logs & API Quota Management [Sprint 7 120h]	Admin	Monitor system-wide platform health, view audit logs, and configure global Gemini AI token limits	I can ensure operational stability, track user usage patterns, and prevent unexpected infrastructure expenses	Medium

2.3 Product Backlog Specification (User)

Table 9 - Product Backlog Specification (User)

ID	Theme	As a/an	I want to	So that	Priority
PB-01	User Registration & Account Management [Sprint 1 36h]	User	Register an account with email, name, and strong password, and verify credentials	I can create an authenticated user profile to securely access the CovAI platform	High
PB-02	User Login & JWT Session Management [Sprint 1 52h]	User	Log in with email and password, receive a signed JWT token, and authenticate API requests	I can securely access protected endpoints and maintain an active session	High
PB-03	Password Recovery & Secure Reset Workflow [Sprint 1 56h]	User	Request a password reset link via email and reset password using a secure one-time token	I can regain access to my account if I forget my credentials	High
PB-04	User Profile & Encrypted GitHub Token Configuration [Sprint 1 28h]	User	View/update my profile details and configure my GitHub Personal Access Token (PAT)	CovAI can authenticate against GitHub APIs to clone private repositories securely	High
PB-05	Platform Navigation & Project Management Workspace [Sprint 1 60h]	User	Navigate Homepage and Landscape overview, and manage projects (create, view, edit, delete)	I can organize repositories, view status overviews, and maintain project configurations	High
PB-06	GitHub Repository Ingestion Setup [Sprint 1 36h]	User	Link GitHub repositories to projects by specifying repository URL and branch	The system can pull remote source code for automated testing and coverage analysis	High
PB-07	Snapshot Management & Core Database Infrastructure [Sprint 1 68h]	User	Create immutable snapshots for source versions and query version histories	I can track code quality over time, compare test runs, and ensure reproducibility	High
PB-08	Local ZIP Archive Project Ingestion [Sprint 2 64h]	User	Upload project source code as a ZIP archive with automated sanitization and safety checks	I can analyze local codebases while protecting the server against malicious archives	High
PB-09	GitHub Repository Shallow Clone Ingestion [Sprint 2 64h]	User	Import remote repositories directly from GitHub via automated shallow cloning	I can analyze hosted repositories without manual packaging or file transfers	High

PB-10	Automated Testing Framework Detection [Sprint 2 64h]	User	Have the system automatically inspect project structure and detect configured testing frameworks	I don't have to manually configure test runners, config paths, and coverage reporters	High
PB-11	Isolated Docker Sandbox Test Execution [Sprint 2 80h]	User	Execute automated test suites inside an isolated, resource-constrained Docker container	Tests execute securely without risking host compromise, dependency pollution, or resource exhaustion	High
PB-12	Multi-Level Code Coverage Analysis [Sprint 2 64h]	User	View comprehensive test coverage metrics across Statement, Branch, Function, and Line dimensions	I can precisely identify which source files and logic paths are tested versus uncovered	High
PB-13	AST Parsing & Interactive Control Flow Graph (CFG) [Sprint 3 64h]	User	Parse JavaScript/TypeScript ASTs and inspect interactive Control Flow Graphs for functions	I can visually trace execution branching and distinguish covered from uncovered execution paths	Medium
PB-14	Cyclomatic Complexity & Testing Gap Prioritization [Sprint 3 56h]	User	Calculate Cyclomatic Complexity for every function and get a prioritized list of testing gaps	I can focus testing effort on high-risk, defect-prone functions with low coverage and high complexity	Medium
PB-15	Multi-Framework Test Runner Adapters [Sprint 3 48h]	User	Execute tests across Unit, Integration, and E2E frameworks (Jest, Vitest, Supertest, Playwright, Cypress)	I can evaluate test coverage and execution health across all layers of the testing pyramid	Medium
PB-16	AI-Assisted Test Skeleton Generation via Gemini API [Sprint 3 56h]	User	Request AI-generated test skeletons for uncovered functions using Google Gemini API	I can quickly get scaffolded test structures, describe blocks, and mock setups for untested code	High
PB-17	AI-Assisted Full Runnable Test Case Generation [Sprint 3 64h]	User	Generate complete, runnable unit test implementations targeting specific uncovered branches	I can directly run generated tests to close coverage gaps without writing boilerplate setup manually	High

PB-18	Generated Test Case Validation & Code Export [Sprint 3 48h]	User	Review, edit, validate in sandbox, and export AI-generated test files into my project	I can verify test correctness and safely integrate new test suites into my codebase	Medium
PB-19	GitHub Webhook Automation & Continuous Analysis [Sprint 4 40h]	User	Configure GitHub repository webhooks to automatically trigger test runs upon new commits or PRs	My project coverage and quality scores are continuously analyzed without manual trigger actions	Medium
PB-21	Real-Time WebSocket Push Notifications [Sprint 4 32h]	User	Receive real-time WebSocket push notifications when long-running analysis or AI generation completes	I don't have to repeatedly refresh the browser or wait idle on the screen during background processing	Low
PB-22	Software Quality Overview Dashboard & Radar Metrics [Sprint 4 32h]	User	View a centralized Quality Dashboard with composite quality scores and multi-dimensional radar charts	I can holistically evaluate software release readiness across coverage, complexity, and maintainability	High
PB-23	AI Evaluation Knowledge Ingestion & Document Parsing [Sprint 5 48h]	User	Upload project business documents, API specifications, FAQ files, and operational rule sets	CovAI can establish the project ground truth required for evaluating internal chatbot trustworthiness	Medium
PB-24	Evaluation Profile Builder & Benchmark Scenario Generation [Sprint 5 56h]	User	Configure target chatbot endpoints and automatically generate candidate evaluation scenarios	I can create a structured benchmark dataset covering Knowledge Q&A, Usage Guidance, and Action/Tool execution	High
PB-25	Domain Expert Rule & Ground Truth Confirmation [Sprint 5 40h]	User	Review, edit, approve, or add ground truth facts, expected UI sequences, and forbidden actions	The evaluation benchmark precisely reflects validated business logic and prevents false positive penalties	Medium
PB-26	Multi-Task Chatbot Evaluation Runner [Sprint 5 56h]	User	Execute automated benchmark evaluation runs against the target chatbot interface	The system systematically tests all approved scenarios and captures raw responses, latency, and metadata	High

PB-27	Chatbot Tool & Action Execution Safety Verification [Sprint 5 48h]	User	Verify tool selection accuracy, parameter schemas, safety confirmations, and forbidden actions	I can prevent risky external mutations and ensure the chatbot safely adheres to API authorization rules	High
PB-28	3-Layer Trustworthiness Multi-Dimensional Scoring Engine [Sprint 5 48h]	User	Evaluate chatbot outputs across the 3-Layer framework (Core Criteria, Task Module, Project Rules)	I receive objective, multi-dimensional scores on Correctness, Faithfulness, Hallucination, Safety, and Latency	High
PB-29	Chatbot Diagnosis, Benchmark Comparison & Report Export [Sprint 5 40h]	User	Inspect detailed failure diagnosis, compare multiple chatbot versions/models, and export benchmark reports	I can identify root causes of chatbot errors, track model improvements, and present compliance evidence	Medium
PB-30	End-to-End System Integration & Regression Testing [Sprint 6 96h]	User	Perform automated end-to-end integration and regression test runs across the complete platform pipeline	I can ensure all interconnected modules (Ingestion -> Sandbox -> Coverage -> CFG -> AI -> Evaluation) work seamlessly	High

3. Constraints

Table 10 - Constraints

Constraint	Condition
Time	Project development duration is 18 weeks (from August 10, 2026 to December 26, 2026), structured into 7 Agile/Scrum Sprints: Sprint 1 (Environment Setup, Authentication & Project Management), Sprint 2 (Source Ingestion, Test Execution & Coverage Analysis), Sprint 3 (Code Analysis, Multi-Test Framework Support & AI Test Generation), Sprint 4 (GitHub Automation, Monitoring & System Integration), Sprint 5 (AI Chatbot Trustworthiness Evaluation Module), Sprint 6 (End-to-End System Testing & Quality Assurance), and Sprint 7 (System Deployment, Final Fixes & Project Delivery), followed by final document submission and Capstone defense.
People constraint	The project is designed, engineered, and maintained by a dedicated team of 4 software engineering students from Duy Tan University (Dinh, Huynh Tan; Thuan, Ngo Huu; Phuc, Tran Huu; Quy, Nguyen Duy) under the academic supervision of Ph.D. Nguyen Duc Man. Total committed development effort is 2,568 man-hours (642 hours per member, based on 6 working hours/day).
Hardware & OS	Development is conducted on personal Windows 10/11 and Linux workstations. Server runtime requires Docker Engine 24+ and Node.js 20 LTS. Test sandboxes execute within <=1 GB RAM, <=1 vCPU, and a 30-second timeout.

Software & Runtime	Client built with React 19 and Vite; API built with Express.js 5 and Node.js 20 LTS; Database is PostgreSQL managed via Prisma ORM; Job queues are powered by BullMQ and Redis; AI capabilities integrate Google Gemini API (gemini-1.5-flash / gemini-1.5-pro).
Economic & AI Budget	Direct project budget is 800,000–1,150,000 VND (approximately 50 USD); student labor is non-billable academic effort. Content hashing (SHA-256) and snapshot caching control repeated analysis and AI token expenses.
Standards & Quality	Development strictly adheres to Product Owner requirements, Agile/Scrum empirical process control, ISO/IEC/IEEE 29119 Software Testing standards, ISO/IEC 25010 Software Quality models, and IEEE 830/29148 requirements specifications.
Safety & Real-World Protection	Evaluation of action-oriented chatbots and dynamic test execution strictly avoids unintended real-world side effects through simulated tool environments, mandatory human confirmation rules for destructive actions, and strict rate limiting.

4. Stakeholders and User Descriptions Summary

Table 11 - Stakeholders and User Descriptions Summary

Name	Description	Role
Product Owner	Provides product vision, defines acceptance criteria, reviews Sprint deliverables, and accepts system increments.	Duy Tan University / Man, Nguyen Duc, Ph.D.
Scrum Master / Project Leader	Leads agile processes, facilitates Daily Scrums and Sprint Planning, removes technical impediments, and ensures timely milestone delivery.	Dinh, Huynh Tan
Frontend Lead / Full-Stack Developer	Architects React 19 SPA, designs responsive UI/UX, interactive CFG graphs, real-time coverage dashboards, and notification components.	Thuan, Ngo Huu
Backend & Integration Developer	Builds RESTful APIs, database schema, BullMQ worker queues, Docker Sandbox isolation environments, and end-to-end integration.	Phuc, Tran Huu
Data & Full-Stack Developer	Develops data analysis pipelines, Babel AST parser, CFG generator, database query optimization, and UI modules.	Quy, Nguyen Duy
AI Evaluator / Domain Expert	Defines chatbot evaluation profiles, validates ground truth rules and expected actions, audits benchmark scenarios, and analyzes trustworthiness scores.	AI Evaluation Specialist / Domain Expert

System Administrator	Manages cloud infrastructure, Docker host environments, BullMQ queues, database backups, security audits, and Gemini API quotas.	C1SE.30 Team
Software Developer (End-User)	Primary user who imports projects, executes automated test suites, inspects coverage/CFG graphs, and generates test cases via AI.	Software Developers & Students
QA Engineer / Tester (End-User)	Evaluates test coverage metrics, traces requirements to tests, prioritizes coverage gaps, audits AI-generated tests, and exports quality reports.	QA Engineers & Testers

5. References

Table 12 - References

No	Reference	Note
1	https://www.scrum.org/resources/scrum-guide	Scrum Guide & Agile Product Backlog Standards
2	https://www.mountaingoatsoftware.com/agile/scrum/product-backlog/example/	Scrum Product Backlog Example & Estimation
3	C1SE.30_Proposal_ver1.2.pdf	CovAI Project Proposal Document (version 1.2)
4	C1SE.30_ProjectPlan_ver_1.1-C1SE.30_Sprint1_COVAI_ver_1.0.pdf	CovAI Project Plan & Sprint Schedule Document (version 1.1)
5	https://ai.google.dev/gemini-api/docs	Google Gemini API Documentation
6	https://www.iso.org/standard/81291.html	ISO/IEC/IEEE 29119 Software Testing Standards
7	https://www.iso.org/standard/35733.html	ISO/IEC 25010 Systems and Software Quality Requirements and Evaluation
8	https://is.duytan.edu.vn/en/programs-curriculum/software-engineering/1/20/course-description-bachelor-degree-in-software-engineering/613	Official CMU-SE 450 course description and staged capstone deliverables